

GhostLink Diagnostic Tool: Design and Build Plan

Overview

The GhostLink Diagnostic Tool is a **standalone, sovereign diagnostics appliance** built on a Raspberry Pi (using the latest Raspberry Pi 5 for maximum performance). It provides three core functions in one modular device: **electrical multimeter diagnostics** (voltage measurement and continuity testing), **mechanical logic assistance** (part tracking and torque specification reference), and **OBD-II vehicle diagnostics** (reading engine data and fault codes over CAN and K-Line protocols). The system is designed to run **offline-first**, with an option to **“burst” online** briefly for AI-based fault interpretation when the user explicitly permits it. It boots directly into a minimal GhostLink OS interface – no desktop environment or bloat – giving a **fast, cold-start overlay** for the tool. The design emphasizes efficiency (can be powered by a USB battery or vehicle OBD port), and is fully **extensible**: it serves as the **runtime core for future GhostLink tool packs**, meaning the architecture anticipates additional modules (e.g. a fabrication pack) that can plug into this base. In summary, this first GhostLink tool is a simple yet powerful diagnostics hub, **purpose-built for sovereignty, modularity, and offline functionality**.

Hardware Platform and Modular Components

Core Board – Raspberry Pi 5: The device centers on a Raspberry Pi 5 (or Raspberry Pi 4 if 5 is unavailable). The Pi provides ample CPU and RAM to handle real-time data processing and optional AI tasks. The choice of Pi ensures modularity (easy to attach HATs or peripherals) and community support. We leverage the Pi’s GPIO and serial interfaces to integrate diagnostics hardware. The Pi 5 runs a 64-bit lightweight Linux OS (GhostLink OS environment) and interfaces with the following modules:

- **OBD-II Interface Module:** For vehicle diagnostics, a dedicated OBD-II interface is attached to the Pi. This can be a **CAN bus controller and transceiver** (for OBD-II CAN protocols) combined with a **K-Line transceiver** for older vehicles. For example, a Pi CAN HAT (such as one based on the MCP2515 CAN controller) can handle CAN messaging, while an ELM327-compatible chip (or STN1110/STN2100 module) bridges the Pi’s UART to K-Line and CAN protocols via standard AT commands. In fact, existing solutions like the DIAMEX Pi-OBd board integrate an *“advanced OBD2 interface and power supply”* for the Pi, allowing access to vehicle data, trouble code reading/clearing, etc. ¹. We will emulate such designs: the Pi’s UART (GPIO serial TX/RX) connects to the OBD interpreter module, which handles low-level protocol timing (5 baud init for ISO9141 K-Line, CAN speeds, etc.) and provides a uniform interface. This modular approach means the OBD hardware can be swapped or upgraded (for example, to support CAN-FD or manufacturer-specific protocols) without changing the core software. The OBD port’s 12 V supply can also be converted (via a DC-DC regulator) to power the Pi, so in-vehicle use might not require a separate battery – although the tool can just as easily run on a USB power bank for portability.
- **Analog/Digital Sensing (Multimeter) Module:** Because Raspberry Pi boards lack analog input, we include an **ADC (Analog-to-Digital Converter) chip** to measure voltages ². A suitable choice is an SPI or I²C ADC (e.g. MCP3008 for 10-bit or MCP3204/MCP3208 for 12-bit resolution). This ADC

connects to the Pi's SPI interface and provides multiple channels for analog measurement (at least one channel for a voltmeter input, and possibly additional channels for future sensors or monitoring battery voltage). We design a simple input circuit for the voltmeter: a voltage divider and protection network that scales the measured voltage into the ADC's 0–3.3 V range, allowing measurement of typical automotive voltages (0–~20 V DC) with overhead for spikes. Protective components (resistors, transient suppression diodes) ensure the Pi and ADC are not damaged by over-voltage or ESD. For **continuity testing**, we use a GPIO-based approach: one GPIO pin configured as a current source (or simply a HIGH output through a resistor) and another as a voltage-sensing input. When the two multimeter probes complete a circuit (low resistance path), the input pin will read LOW (near 0 V); if the circuit is open, the input reads HIGH. This effectively checks if resistance is below a threshold (a few tens of ohms) indicating continuity. A small buzzer (or piezo speaker) is connected to another GPIO so the tool can emit an audible beep when continuity is detected, mimicking a DMM's continuity tester. By combining the ADC readings and GPIO logic, the tool covers basic multimeter functions: measuring DC voltage and continuity/ohms. These hardware components are modular – e.g. the ADC could be swapped for a higher-resolution one if needed, or an external microcontroller could be added later to handle more advanced measurements – but the initial design keeps it simple with off-the-shelf parts. *(Note: The Pi's lack of built-in ADC necessitates this extra component; alternatives like using an Arduino as an ADC were considered, but an on-board ADC IC is simpler and keeps the design self-contained ².)*

- **Touchscreen Display:** The user interface is presented on a small **touchscreen** attached to the Pi. A 5"–7" display (such as the official Raspberry Pi 7-inch touchscreen at 800×480 resolution) is sufficient. We run the UI in fullscreen mode on this display with no window manager. Touch input allows intuitive control in the field (gloved use might require a stylus or bigger UI elements, which we account for in design). The touchscreen connects via the Pi's DSI connector or HDMI/USB (depending on the model). This display is also a modular part – the system could support different screen sizes or even headless operation with output through SSH or web (for advanced users), but the primary mode is the self-contained touchscreen GUI.
- **Power Supply:** The tool is **power-agnostic** in that it doesn't rely on a custom battery pack. It can be powered via the Pi's 5 V microUSB/USB-C input from any **USB battery bank** or adapter. When used in a vehicle, it could also be powered directly from the OBD-II port's 12 V (which we step down to 5 V). In the initial build, a simple approach is to use a car USB adapter or the Pi-OBd board's built-in regulator to power the Pi from the car's OBD port ³. The system is optimized for **low power draw** when possible – turning off unused interfaces (Wi-Fi, etc.) when not needed and using the Pi's power-saving features – to maximize battery life.

All these components are arranged to maintain **modularity and future expansion**. The Pi's GPIO header still has free lines for additional modules (e.g. connecting a future fabrication tool controller or additional sensors). The case design will allow adding or replacing modules (for example, a slot for a different HAT or a connector for an external device). In essence, the Raspberry Pi acts as a **hub**, and each diagnostic function is a plug-in module (ADC board, OBD interface board, etc.) that can be developed or upgraded independently. This modular hardware layout reflects GhostLink's ethos of customizability and upgradability.

Boot and OS Behavior (GhostLink OS Mode)

The device boots directly into **GhostLink OS mode**, meaning the diagnostics application is up and running immediately after power-on, without any manual steps. We base the system on a minimal Linux (for instance, Raspberry Pi OS Lite or a custom Buildroot image) that has only the essential drivers and our GhostLink software. The boot process is configured as follows:

- **Optimized Boot Sequence:** Unnecessary services (network daemons, GUI login managers, etc.) are disabled to reduce boot time and resource usage. As soon as the kernel initializes the needed hardware (display, touchscreen, SPI, UART, etc.), the system launches our GhostLink tool. We achieve this by using a systemd service or an `rc.local` script that auto-starts the GhostLink main program. The Pi is set to auto-login on the console with no password and immediately execute the application. There is **no desktop environment** loaded at all – this is effectively a kiosk/embedded setup where our app is PID 1 or launched by init. (In Raspberry Pi terms, this is analogous to running in kiosk mode where the system *“boots straight into a full-screen application without using the desktop environment.”* ⁴) This not only speeds up boot (the user sees the interface quickly) but also aligns with security (fewer background processes running).
- **Direct-to-UI Startup:** Upon boot, the user is greeted with the GhostLink Diagnostics UI on the touchscreen, without any Linux login prompt or shell visible. We suppress the usual boot text on HDMI/DSI output (by editing `cmdline.txt` and using a splash screen or blank screen). We may display a simple GhostLink logo or “cold start” indicator during the 2-3 seconds of boot, but otherwise it transitions straight to the tool’s main menu. Essentially, the Pi behaves like a dedicated appliance – insert power, and it **boots into the tool** automatically. If needed for debugging, one can still SSH in or attach a keyboard and switch to a TTY, but by default the user never sees a Linux console.
- **Minimal OS Environment:** GhostLink OS mode is a **“cold” execution environment** – stripped of background internet-connected services and unpredictable behaviors. There are no cloud sync daemons, no package auto-updaters, nothing running that isn’t absolutely required. This deterministic startup ensures the same behavior every time (important for a diagnostic tool). The OS is configured with required kernel modules: e.g. SPI driver for ADC, I2C if used, CAN kernel driver (SocketCAN) if using a CAN controller, and the UART enabled for OBD. Everything else is either removed or left inactive. Networking is kept **offline by default** (Wi-Fi radio off or not connected, to prevent any background network activity – more on that in the Grant section). This adheres to GhostLink’s “air-gapped by default” principle for security. The filesystem can be set mostly read-only or journaled to avoid corruption on sudden power loss (since field tools might be unplugged without clean shutdown). Logs and the Vault data can reside on a separate partition or in-memory until saved, to further protect the core OS image from damage.
- **Autostart and Process Supervision:** The GhostLink main application is launched as the sole UI process. We can run it directly on the Linux framebuffer (using SDL or similar) or within a lightweight X/Wayland session if a GUI toolkit requires it – but without a window manager. The app runs full-screen; if it crashes or exits, a watchdog can restart it or drop to a maintenance console. We want a robust appliance feel, so we might use systemd to respawn the service if it dies unexpectedly. The user does not have the ability to accidentally exit to the OS; a special command or physical key (hidden in UI) would be needed to quit the app, ensuring the device stays in the tool interface at all

times. Essentially, GhostLink OS mode locks the Pi to *only* run GhostLink tools, reinforcing the idea of a dedicated instrument.

- **Boot Behavior Example:** When power is supplied, the Pi's bootloader brings up Linux, which mounts the root filesystem (possibly from SD or an embedded storage). A custom boot splash (GhostLink logo) is shown or the screen remains blank. Within a few seconds, the `ghostlink.service` starts: this might run a script that initializes any needed GPIO modes (e.g. set up ADC chip select pins, turn on CAN interface via `ip link` commands, etc.) and then launches the **GhostLink Diagnostics application**. The application might perform a quick self-check (e.g. ensure it can read from the ADC, confirm the OBD interface is responding on the serial port, etc.). Then it displays the **Main Menu** on the touchscreen. The entire process from cold boot to menu could be on the order of ~10 seconds on a Pi 5, though initial builds might be within ~20 seconds. From the user's perspective, they turn it on and it goes right into the diagnostic tool – no OS mucking about.

This boot arrangement exemplifies the GhostLink philosophy of “**instant readiness**” and **deterministic behavior**. The system always starts in a known good state (freshly loaded software, known vault data) and does not carry over any unpredictable state between sessions (unless intentionally saved in the Vault). It's akin to a *cold metal* reboot each time, avoiding issues of lingering memory. In combination with an **immutable base OS** and all user data in Vault, we could even have the device restore to a known image at each boot for reliability (though that may be an advanced feature – for now, we just ensure a consistent startup sequence).

Application Structure and File System Layout

The GhostLink diagnostics software is organized into a clean, modular file structure on the device. This makes it easy to maintain and to add future expansions (packs). The **directory layout** could be as follows (all residing, for example, under `/opt/ghostlink` or `/home/pi/ghostlink` on the Pi):

- `ghostlink_main.py` (or compiled binary): The main launcher and orchestrator of the tool. This script initializes the UI, handles high-level navigation, and invokes the appropriate module (electrical, OBD, mechanical) based on user input. It also interfaces with the Vault and manages permission grants. If the project is in C/C++, this would be the main program; in Python, it's the entry point that imports all other modules.
- `modules/` **directory:** Contains separate modules for each major function:
 - `electrical.py` – Module for electrical diagnostics. Implements routines to read from the ADC, interpret continuity, and format the data for display (voltage reading, etc.).
 - `obd.py` – Module for OBD-II diagnostics. Handles connecting to the vehicle's ECU, sending OBD requests (via serial or CAN), and parsing responses (trouble codes, sensor values). Could use a library (like Python `OBD` module) internally for convenience ⁵, or implement low-level SocketCAN messages if using CAN directly.
 - `mechanical.py` – Module for mechanical logic assistance. Manages data structures for part tracking (lists of parts/steps) and torque specifications (lookup or storage). Also could handle any user input for notes.

- (Future packs can add more modules in this folder, e.g. `fabrication.py` for a fabrication pack's functions, without interfering with existing ones.)
- `ui/` **directory**: Defines the user interface layout and resources. This might include:
 - `ui_layout.json` or `ui.py` – descriptions of screens, menus, and widgets. If using a UI library, this could be code that constructs buttons, text fields, etc.
 - Icon assets or small images (if any icons are used for buttons, though we aim for text-based minimalism so this might be very sparse).
 - Styles or theme configurations (e.g. specifying color scheme: GhostLink might use a dark background with high-contrast text).
 - Localization strings if needed (probably not initially; default to English).
Essentially, all visual and interactive elements are encapsulated here so that changing the look/feel doesn't require altering core logic.
- `vault/` **directory**: This directory (or set of files) contains the **Vault and knowledge base** for the tool. It includes:
 - **Persistent data files** capturing state or knowledge. For example:
 - `memory.vault` – (If using GhostLink's style) a file storing any persistent agent memory or recent interactions (for the AI assistant, if applicable). In our cold-first version, this may simply log recent fault interpretations or notes.
 - `sensor_trace.vault` – logs of sensor readings or test results. The GhostLink architecture analysis shows sensor_trace vaults store diagnostic traces from hardware inputs ⁶. Our tool can store, for instance, a history of voltage readings or OBD data captured during a session (timestamped), so that this data can be reviewed later or fed into an AI analysis.
 - `dtc_codes.json` or `dtc.vault` – a database of Diagnostic Trouble Code definitions (e.g. "P0301: Cylinder 1 misfire detected"). We will include a compiled list of standard OBD-II codes and their plain-English descriptions. This lets the device explain a code without internet. (Many standard codes are well-known; we can hard-code or store them here for quick lookup.)
 - `torque_specs.json` – a user-editable file containing torque specifications. Initially this might be empty or have a few examples, but users (or future packs) can populate it with the values relevant to their equipment (e.g. "Wheel lug nut – 100 Nm"). The mechanical module reads this to display recommended torques.
 - `parts_log.txt` – a log file or small database that records parts removed/replaced in a session, so a user can review what was done. This could simply be written each time the user marks a part in the UI.
 - `ghostlink.db` – optionally, a SQLite database to store structured logs, user settings, and session transcripts. For example, it might have a table of past OBD sessions and what codes were found, or store the user's preference for units (metric/imperial). The GhostLink research indicates use of an SQLite DB for logging interactions in some prototypes ⁷, which we can mirror for robust data handling.
 - **Macro definitions** (for GhostLink's scripted tool sequences): e.g. `macros.vault`. This file can define any automated diagnostic sequences or multi-step procedures. In the initial version, we might have a few basic macros (like a macro to run a "full diagnostic test" by checking battery

voltage, reading OBD codes, and cross-referencing known issues). Each macro is a sequence of actions. The GhostLink system uses vault files for macros in a human-readable format ⁸ ⁹. We'll integrate that concept so that advanced users or future packs can add new macros without changing code. (For instance, the *Electrical Pack* vs *Fabricator Pack* might ship with different macro sets tailored to those domains ¹⁰.) For now, macros are minimal, but the infrastructure is there.

- The Vault directory (and any `.vault` or `.json` files) can be synced or backed up by the user. It is the persistent “memory” of the tool, as opposed to the stateless core code.

• **Config and Integration Files:** These include any system integration scripts:

- `ghostlink.service` (in `/etc/systemd/system/`) – the service file that starts the main program at boot (as discussed).
- `ghostlink_boot.sh` – if needed, a script run at boot to perform any shell-level setup (like configuring CAN interface: e.g. `ip link set can0 up type can bitrate 500000` for CAN bus at 500 kbps, enabling SPI if not already, etc.). This could also load Vault data into memory or set environment variables (like an OpenAI API key from a secure file in the vault).
- `settings.cfg` – a simple config file (could be in the vault or `/etc`) for user-tunable settings, like default units, display brightness, etc.

- **Security keys** – If there are API keys or cryptographic keys (for example, a key to authenticate pack updates or an OpenAI API token), these are stored in a secure manner in the vault (possibly encrypted or encoded). For now, the user might manually place their OpenAI API key in a text file in the vault (`openai.key`) which the software will read when it needs to query the API. This keeps secrets off the code and in the user-controlled storage.

- **Logs:** While some logging goes into the vault or database, we may also use standard log files for system diagnostics (e.g. application debug logs in `/var/log/ghostlink.log` or similar) for troubleshooting the tool itself. These can be rotated or cleared on each boot since we want a memoryless feel (GhostLink tends to not accumulate state unless intended).

This file structure ensures a **clean separation** of concerns: UI vs logic vs data vs system integration. It will be packaged such that deploying the tool is straightforward (just copy the ghostlink directory and install the service). It also sets the stage for bundling the tool – we can zip up the entire `/opt/ghostlink` folder and that constitutes the “GhostLink Diagnostics Pack”. This pack could be delivered as a bundle for others to use or as the base onto which further packs will be layered.

Notably, by structuring vault data and tool modules this way, the core **GhostLink runtime** (UI framework, permission system, etc.) remains constant, and **additional tools (packs)** can be dropped in as new modules with their own vault extensions. For example, if a Fabrication pack adds a `fabrication.py` and some macro entries in a vault file, the main program can dynamically detect and include it in the menu. This dynamic loading could be done via a simple plugin registry – e.g., the main script scans `modules/` for any new modules and reads a descriptor (maybe a comment or a small manifest each module provides) to populate the UI. That way, no hard-coded changes are needed to integrate a new pack's features. This design echoes GhostLink's modular “toolbox” concept where each tool is an independent piece of code and a *capability* that can be invoked by name ⁹.

User Interface & Interaction Design

The GhostLink Diagnostic Tool features a **minimalistic touchscreen UI** that emphasizes clarity and ease of use in the field. The interface adheres to the principle of “*no gratuitous GUI elements*” – it shows only what the user needs for the task at hand, with no clutter ¹¹. The look is deliberately simple: a dark background with high-contrast text (reminiscent of a terminal or vintage diagnostic equipment). This not only aligns with the GhostLink aesthetic (“cold metal” vibe) but also improves readability outdoors or in various lighting. The UI is navigable via touch (finger or stylus), and also supports physical keyboard input as a fallback (useful if the user plugs in a keyboard or for accessibility). Below is the UI layout and logic:

- **Main Menu:** Upon boot, the main menu is displayed. It likely consists of large, touch-friendly buttons or list items for each major function: **Electrical Diagnostics**, **OBD-II Diagnostics**, **Mechanical Tools**, plus perhaps **Settings** or **About**. The menu may be text-based (e.g. a list of options that can be tapped) but styled with clear delineation (borders or highlights) to indicate touch targets. Only a handful of top-level options are shown to avoid confusion. The design follows the idea of a unified command interface: everything stems from this one screen when idle. (In a more advanced GhostLink UI concept, this could also include a text **command prompt** at the bottom for expert users to type commands instead of tapping – for example, typing “OBD” to jump to that tool – blending CLI and GUI ¹². This “DreamShell” hybrid approach is considered, but in the first version we keep it simple with just touch selection, while laying groundwork for command parsing in the future.)
- **Screen Layout:** Each tool screen (Electrical, OBD, Mechanical) uses a consistent layout template for coherence. There is a header area that displays the mode (e.g. “ Electrical Diagnostics” or “ OBD-II Scanner”) and a **status bar** with small icons for system status (battery level if detectable, an offline/online indicator, etc.). We avoid multi-window or multi-tasking interfaces – only one screen is shown at a time, occupying full screen, to reduce cognitive load. A footer or corner might contain a “Back” button (to return to main menu) and perhaps a “Home” icon to jump to main menu quickly. Navigation between screens is linear and obvious (main menu → sub-tool → possibly sub-functions within tool, then back). There are no overlapping dialogs except simple pop-up confirmations (which cover the center of the screen with a message and yes/no buttons).
- **Touch Interaction:** All buttons and list entries are sized for finger input (at least ~1cm height on screen, with appropriate spacing). The UI avoids requiring fine scroll or pinch gestures – for instance, lists scroll with large arrow buttons if needed rather than tiny scrollbars. In electrical readouts, values are large and easy to read. Interactive elements are kept to a minimum to focus on output data (e.g., the voltmeter screen mostly just shows the voltage reading in big digits). The color scheme is largely monochrome (bright text on dark background) with maybe one accent color (for example, green or amber) to denote active statuses or warnings. This is in line with Nielsen’s aesthetic minimalism heuristic – “*Every extra unit of information in an interface competes with the relevant units of information*”, so we strive to show only relevant info ¹¹. For example, on the OBD screen we wouldn’t show decorative graphics of a car; we’d show the data like “Engine RPM: 3000” plainly, as that’s what the user needs.
- **Electrical Diagnostics UI:** When the user selects this mode, the screen presents a **voltmeter/continuity interface**. The design is reminiscent of a multimeter’s display: a large numeric readout in the center for the voltage reading (e.g. “12.45 V”). If the reading is stable, it updates a few times per

second. If the value exceeds certain thresholds (like >15V or <11V in automotive context), it could change color or trigger a note (e.g. red text if voltage is outside normal range). A small indicator icon shows if continuity mode is active. The user can toggle between “Voltage” and “Continuity” via a button. In continuity mode, the numeric display might be replaced or supplemented with a bold **“CONTINUITY”** indicator when a connection is detected (and we sound the buzzer). We can also display the measured resistance or voltage drop across the probes if we have that data – e.g. “0 Ω – Closed Circuit” when shorted, or “OL – Open Circuit” when not. The UI here is minimal: essentially one big piece of data and perhaps a couple of buttons (toggle mode, hold reading, back). This simplicity ensures quick reading without distraction, important when a user is, say, probing wires and glancing at the screen.

- **OBD-II Diagnostics UI:** Upon entering, the tool will attempt to connect to the vehicle’s ECU. The UI might show a status message “Connecting to ECU...” for a second. Once connected, it shows a **menu of OBD functions:** for example, **Read Current Data, Trouble Codes, Clear Codes, Scan VIN/Info.** We might implement it as a list of options on the left side and a data display area on the right side (especially if we use a wider screen). If the user taps “Read Current Data”, the system can query a few key engine PIDs (like RPM, coolant temperature, etc.) and display them in a list or gauge format. This can be basic at first: e.g. a text list:

- Engine RPM: 3000 rpm
- Coolant Temp: 90 °C

- Vehicle Speed: 0 km/h

(We can fetch these via standard OBD-II PID commands). The UI would update these in near-real-time (maybe one update per second or on user refresh, to keep things light). If “Trouble Codes” is selected, the tool sends a Mode 03 request to get stored DTCs. The screen then lists any error codes, e.g. “P0301 – Cylinder 1 Misfire Detected”. Thanks to our local code database, we can show the text meaning immediately. If multiple ECUs or modules are present (some cars have separate transmission ECUs, etc.), we might list codes by module, but for simplicity we focus on the main powertrain codes. Each code listed could have a small “More info” button. If the user taps a code, we can show a detail view: possibly including freeze frame data or a brief description. Here’s where the **AI interpretation option** comes in: on the code detail view, there may be a button **“Explain/ Diagnose”**. Pressing this will prompt the system to prepare a query for an AI (e.g., OpenAI) including the code and any context (vehicle info, sensor readings) to get a human-friendly explanation or troubleshooting steps. When the user taps it, the UI will first show a **Grant prompt** (since going online is required – see Vault & Grant section for logic) and then, if granted, indicate “Querying AI...”. After a moment, the AI’s response is displayed below the code – for example: *“This code indicates a misfire in cylinder 1. Common causes include a faulty spark plug or ignition coil. Check if the cylinder’s spark plug is worn or the coil pack needs replacement.”*. This text is shown in a scrollable text box or wrapped on screen. The user can then acknowledge and perhaps save this explanation to the Vault (e.g. attach it as a note to that code for future reference).

The OBD UI also includes a **“Clear Codes”** function. If selected, a confirmation dialog appears (“Are you sure you want to clear DTCs? Engine must be off. [Yes]/[No]”), since this sends a reset command. On yes, the tool sends the command and then updates the code list (likely “No trouble codes stored” if successful). The UI may then instruct the user to cycle the ignition if needed.

Another sub-function might be **Vehicle Information** (OBD Mode 09) to show the VIN, calibration ID, etc. This can be a simple text output.

Overall, the OBD interface is designed to cover the essential diagnostic tasks a mechanic or DIYer

would do with a handheld scanner: read live data, check/clear fault codes, and view vehicle info. We avoid extraneous features in UI (like graphing O2 sensor data over time) in this first version, though we plan the data logging so that such analysis can be done or added later. The UI remains text-focused because that's the clearest way to convey this info on a small screen. (Our design philosophy is closer to classic OBD handhelds which often had simple character LCDs, but we augment that with the ability to show longer text like AI explanations or multiple lines at once thanks to the touchscreen.)

- **Mechanical Tools UI:** This section assists with physical repair logic, and its interface is more free-form (since it's not reading from hardware as much as providing reference information). We break it into two main parts: **Part Tracking** and **Torque Specs** (the UI may present these as two tabs or buttons within the Mechanical screen).

- *Part Tracking:* The UI here can be thought of as a checklist or notepad. For example, if the user is disassembling something, they can tap "Add part" and enter a name or choose from a list of common parts. The screen could show a list like:

- [] Air Filter
- [] Spark Plug (Cyl 1)
- [] Spark Plug (Cyl 2)
- [] ... etc.

Each item has a checkbox (or just a status indicator). The user checks it when that part has been removed or addressed, and later can uncheck or mark it done when reinstalled. The idea is to keep track of loose parts or steps to ensure nothing is forgotten during reassembly. Because typing on a small touchscreen is cumbersome, we can provide a set of common part names in a dropdown or allow a small on-screen keyboard to pop up for naming. We keep this interface simple: it's essentially a text list that the user can modify. This list can be saved to the Vault (e.g. saved as a timestamped log of "Parts removed in session on 2025-08-08: Air Filter, Spark Plugs 1-4"). In future, more elaborate workflow integration (like loading predefined lists for specific repair tasks) could be added, but initially it's user-driven.

- *Torque Specifications:* The UI for torque specs is likely a simple reference lookup. It might have a search or a menu for categories of fasteners. For example, the user could select "Engine -> Spark Plugs" or just search "spark plug" and the tool will display "Spark plug torque: 20 Nm" (just an example). Similarly "Wheel Lug Nut: 110 Nm" etc. We will populate some generic values for demonstration (like common ranges for generic bolts of certain sizes) and strongly encourage the user to update or verify the values for their specific vehicle. The UI could also allow the user to input a new torque spec entry: e.g. they input a label and a number + unit, which then gets saved. The interface for this could simply be two text fields (with on-screen numeric keypad for the number) and an "Add" button. Once added, it appears in the list. Over time, the user can build their own mini-database of torque specs in the device. We will store these in the `torque_specs.json` in the Vault so they persist.

In both sub-sections, the emphasis is on assisting the user's **logical process** rather than measuring something. We make the UI **straightforward and reliable** – essentially digital pen-and-paper for notes and a pocket reference. For example, the part checklist screen might also allow the user to jot a short note for each part (like "location" or "note: needs replacement"), but we'll likely start with just a binary check/uncheck. The visual style remains consistent: text lists, checkboxes, maybe slight color highlights for checked items or warnings for unchecked critical items. It's all offline and local. This mechanical UI can be expanded in the future with more intelligence (for instance, an AI that

suggests a checklist for a procedure, or AR guidance, etc.), but the first version sticks to basics that **bundle knowledge for offline use**.

- **Settings and System UI:** We include a simple Settings page reachable from the main menu (or via a gear icon). This page lets the user configure a few things: e.g. toggle Wi-Fi on/off (with warning that turning it on will enable connectivity), adjust screen brightness, sound volume (for the buzzer feedback), and view system info (maybe the GhostLink version, storage usage, etc.). Also, crucially, a “Shut down” button would reside here to allow a graceful shutdown of the Pi (since just pulling power can risk SD card corruption). The Settings UI is minimal; it might be text options or a basic form. We do not expose standard Linux settings – just what’s relevant to the appliance usage. If the user needs to do OS-level config, they likely will connect via SSH or attach keyboard; average users will not need that.
- **Visual Style and Feedback:** Throughout the UI, we use consistent fonts and symbols. For example, we might use a simple sans-serif or a faux-terminal font for that retro tech feel. Important messages (like “No ECU connection” or “Voltage out of range!”) are displayed prominently, possibly blinking or in a different color to catch attention. All actions that have a potentially destructive effect (clearing codes, toggling network, shutting down) will prompt for confirmation, typically by a popup dialog with an explicit yes/no as mentioned. This ensures the user doesn’t accidentally do something harmful without realizing (consistent with GhostLink’s intent enforcement approach). Where applicable, we use universally understood icons: e.g. a Wi-Fi symbol with a slash to show offline mode, a battery icon if we can detect power state (if on USB power we can’t read battery, but on OBD power maybe we could infer if engine is on/off by voltage; however, that might be an advanced touch, so likely just a dummy battery icon if on external power).

In summary, the UI aims for a **“one task, one screen”** paradigm, **high contrast, and minimal clutter**, following both our usability goals and GhostLink’s design philosophy of functional minimalism. By keeping it simple, we ensure the tool is **usable in a garage environment** (where time and attention are limited) and that it remains stable (fewer moving UI parts mean fewer bugs). The design also inherently supports extension: adding a new tool would be as simple as adding a new menu item and creating a new screen for it with similar structure. This could be done without redesigning the whole interface, maintaining a uniform user experience across the GhostLink ecosystem.

Electrical Diagnostics Module (Multimeter & Continuity)

The electrical diagnostics functionality effectively turns the GhostLink device into a basic **digital multimeter**. Here’s how it’s designed and implemented:

- **ADC Integration:** We use the external ADC (e.g. MCP3204/MCP3008) connected via SPI to measure analog voltages. During system init, the `electrical.py` module sets up the SPI interface (ensuring the Linux SPI driver is enabled and accessible at, say, `/dev/spidev0.0`). The ADC is read periodically in a loop or on-demand when the Electrical screen is active. The ADC channel 0, for instance, is wired to the main voltage measurement circuit. We calibrate it by using a known reference (the ADC’s reference voltage and resistor divider define the scale). For example, if using a 100kΩ:10kΩ divider, a 12 V input yields about 1.09 V at ADC, which the software will scale back to display 12.0 V. Calibration factors (gain and offset) can be stored in a config (perhaps in the vault or code). The ADC reading code likely uses an existing library or a simple bit-banging read – it’s

straightforward to poll an SPI ADC in Python or C. We'll aim for a sampling rate of e.g. 5–10 samples per second for a stable reading (and perhaps take an average to smooth noise).

- **Voltage Measurement Range:** The hardware range might be, say, 0–20 V DC as designed (with the divider and ADC 3.3 V limit). We ensure that even if the user accidentally probes a higher voltage, protection (like a TVS diode and resistor limiting input current) will prevent damage. The software will saturate at max reading (e.g. show ">20V" if out of range). We also account for negative voltages (if leads reversed) – either block them (diode clamp to 0) or measure if using a differential ADC. Simpler: assume user will observe polarity. The UI can show a "-" sign if negative detected, but in automotive context negative isn't common except maybe if leads swapped.
- **Continuity Detection:** For continuity, the design uses a GPIO pin sourcing ~3.3 V through a series resistor (like 1 k Ω) to one probe, and another GPIO pin (or ADC channel) reading the voltage at that probe. If the probes are connected across a low-resistance path (like both touched together or via a wire), the voltage at the sense pin will drop near 0 V (current flows through the resistor to ground). The software can interpret a reading below a threshold (e.g. <0.5 V or so) as "continuity". If using a digital GPIO input with an internal pull-up, it's even simpler: connect one probe to a digital input with pull-up, the other probe to ground – a closed circuit will bring the input low. This threshold-based approach mimics how a real multimeter beeps for low resistance (often <50 Ω). The module will likely allow some *hysteresis* or delay to avoid flicker (so a momentary disconnect doesn't stop the beep instantly, we might require a few consecutive readings to go false before turning off the continuity indicator). When continuity is detected, the module triggers the buzzer GPIO to emit a tone. This could be done by simply toggling a pin or using PWM for a steady tone. The buzzer should be relatively quiet or configurable, to not annoy if not needed – a setting to mute it could be in Settings.
- **Measurement Display & Data Handling:** The electrical module continuously updates a variable with the latest measurement. The UI layer in electrical screen reads this and updates the displayed value every ~0.2s. If the user switches to another screen, the module might pause polling to save CPU (though ADC polling is negligible load). If the user presses a "Hold" button (if provided), we freeze the display at the last value by simply not updating it (but we continue reading in background perhaps). We might also implement "min/max capture" where the module tracks the lowest and highest readings seen since entering the screen, which can be useful for certain diagnostics (this could be shown on UI if space, or on demand). These values can be reset when leaving the screen.
- **Safety Considerations:** The multimeter functionality is basic and not meant to replace a high-end DMM for all scenarios, but for automotive 12 V and continuity it suffices. We'll mention in documentation that it's for low-voltage DC only (since touching mains AC would be dangerous and outside scope). Efficient power use: reading the ADC is low-power; the module uses minimal CPU (just SPI communications). The ADC itself draws little current. We ensure the ADC and probe circuits don't significantly drain the battery when idle; for example, if the probes are not connected, only a tiny leakage flows through the pull-up resistor. In continuity mode, the 1 k Ω resistor at 3.3 V means ~3.3 mA flows in a short – this is fine and won't damage anything (and is safe for most circuits, not enough to bias sensitive electronics strongly).
- **Integration with Vault:** The electrical module can optionally log readings to the vault if a macro or user triggers it. For instance, a macro could sample battery voltage over 60 seconds and save it to `sensor_trace.vault` for analysis (like checking for a voltage drop over time). On each boot or

each start of electrical mode, we might reset any transient data. Persistent calibration or user preferences for this module (like chosen units, e.g. V vs mV, or whether to auto-beep on continuity) can be stored in the Vault config.

- **User Workflow Example:** The user selects Electrical mode. They connect the probes to measure, say, the car battery. The screen immediately shows ~12.6 V. If they crank the engine, they might see it dip or the alternator raise it to ~14.0 V – the screen updates live, giving them feedback. If they want to test a fuse continuity, they switch to Continuity mode with one tap; now when they touch the probes across a fuse, the device beeps if the fuse is good (closed circuit). This immediate feedback, both visual and audio, replicates the multimeter experience. They can then go back to main menu or directly power off the device if done (the latter we discourage without shutdown; ideally they go to Settings -> Shutdown).

In short, the Electrical diagnostics module is **simple, reliable, and always available offline**. It leverages the Pi's GPIO and an ADC to cover the two most common electrical tests (voltage and continuity). By using straightforward threshold logic and averaging, it provides stable readings. This capability, combined with the OBD scanner, means the GhostLink tool can replace carrying a separate multimeter – a big convenience in the field.

OBD-II Diagnostics Module (CAN/K-Line Interface)

The OBD-II module is one of the core features, enabling the GhostLink tool to read vehicle data and fault codes like a professional scan tool. The design covers hardware interfacing, protocol handling, and user-level functionality:

- **Hardware & Protocol Support:** The device connects to the vehicle's OBD-II port (SAE J1962 connector) via the integrated interface board. Our interface handles both **CAN bus** (the modern OBD-II protocol on most post-2008 vehicles) and **K-Line (ISO 9141-2 / KWP2000)** for older vehicles. If using an ELM327/STN1110 chip, that chip internally manages the different protocols and presents a unified serial command set to the Pi. In that case, our software simply opens a serial port (e.g. `/dev/ttyS0` or USB tty) at the appropriate baud (typically 38400 or 115200 for ELM) and issues "AT" commands to configure and query. The advantage is simplicity – we don't need to implement low-level packet timing; we rely on the firmware of the OBD interface. This matches what many Pi-based OBD projects do. For instance, the Python `python-obd` library can "auto-connect to USB or RF ports" using ELM327 commands and then let us query standard OBD data easily ⁵. We will leverage such a library or model our code after it. If using a SocketCAN approach (with a CAN HAT), the software will use Linux SocketCAN sockets to send diagnostic CAN frames (e.g. 0x7DF) and listen for responses (0x7E8/7E9 etc.), implementing the OBD-II PID protocol (ISO-TP for multi-frame data). This is more involved, but it's doable if we avoid the ELM chip. Either way, the module first **detects protocol**: if the car responds on CAN (fast initialization), we proceed with CAN; if not, we try K-Line 5-baud init via the ELM chip or by bit-banging if we had a direct K-Line transceiver. Using a proven module (ELM327 or the DIAMEX DXM as referenced) simplifies this, and since the question suggests K-Line support, we will assume an ELM-compatible interface is present. (The Elektor project example detailed connecting the DXM OBD module's serial lines to the Pi's GPIO UART ¹³ – we essentially follow that pattern: Pi UART <-> OBD module.)

- **Querying OBD Data:** Once the connection is set up, the module can perform various queries:

- Reading **live sensor data** (OBD-II Mode 01 PIDs). For example, engine RPM is PID 0x0C. If using `python-OBDD`, we can simply do `connection.query(obd.commands.RPM)` and get a value in RPM ⁵. Under the hood, this sends the CAN/K-Line message and parses the result. For initial implementation, we will support a handful of common PIDs: RPM, vehicle speed, coolant temp, intake air temp, battery voltage (if supported as PID), etc. The user can select which ones to view, or we show a default set.
- Reading **Diagnostic Trouble Codes** (DTCs). This corresponds to OBD Mode 03 (and Mode 07 for pending codes). The module sends the request and retrieves codes like PXXXX. We then look up each code's description from our local database. If a code is manufacturer-specific (P1XXX, U1XXX, etc.) and not in our DB, we at least display it and perhaps indicate "Manufacturer-specific code – see manual." The list of codes is then passed to the UI for display.
- Clearing codes (Mode 04) as discussed, which doesn't retrieve data but instructs the car to clear error codes and reset MIL. We handle the response (which is usually just an acknowledgment) and update the UI accordingly.
- Other services: Mode 09 for VIN and ECU info. We can retrieve the VIN and display it (as a confirmation of connection and to log which vehicle we scanned – could be saved in the Vault log). We could also fetch Mode 01 PID 0x01 (number of DTCs and MIL status) to show an overall status (e.g. "MIL On, 2 codes stored" might be shown on a summary line).
- We will not delve into advanced OEM-specific diagnostics or bi-directional controls in this first tool; we stick to the OBD-II standard commands that are safe and universal. This keeps complexity down and avoids any risk of altering vehicle state beyond clearing codes.
- **Data Parsing and Presentation:** The module translates raw responses into human-friendly terms. For instance, a Mode 01 PID response gives hex bytes that need scaling. The `python-OBDD` library does this conversion for us (e.g. speed comes as km/h, coolant temp as °C, etc., using standard formulae). If we implement ourselves, we will embed those formulae (which are documented in OBD-II standards). We ensure units are consistent (and allow unit switching in settings, e.g. °C/°F, km/h/MPH). For DTCs, parsing involves reading each 2-byte code and formatting as P0xxx – again straightforward. We also handle the case of "no codes" gracefully. The parsed data is then stored in memory and fed to the UI for display. If the user requests continuous refresh of live data, we might spawn a background thread to poll those PIDs every second and update a data structure that the UI reads. Alternatively, we only fetch on-demand when user selects a specific parameter to avoid flooding the vehicle's bus with requests.
- **Fault Interpretation (AI Integration):** One standout feature is the optional AI assistant for interpreting fault codes or symptoms. This is how it works: When the user chooses to get an explanation for a DTC or a combination of conditions, the OBD module (or a higher-level GhostLink service) will compose a query containing relevant info. For example: "Vehicle: 2010 Toyota Camry, Code P0301 – Cylinder 1 Misfire. Freeze frame: RPM 1500, Load 20%. What are possible causes and fixes?" This query is sent to an online AI service (OpenAI API or similar) **only after the user grants permission** (see Grant section). The module handles the HTTP API call using the Pi's network stack – it might use a Wi-Fi connection or tethered phone for internet. The AI's response is received as text. We parse it (maybe format into bullet points if needed) and pass it to the UI to display under the code details. This effectively gives the user an expert's insight on the fault, supplementing the basic code definition. For instance, the AI might say "Common causes: faulty spark plug, bad ignition coil, or compression issues. First, swap the spark plug with another cylinder and see if the misfire follows..." etc. This kind of guidance can be extremely helpful to a user in the field who might

otherwise need to search manuals or forums. By integrating it on-demand, we **keep the device offline majority of the time**, yet provide a powerful knowledge boost when connected.

We design this process to be as seamless as possible: The user taps “Explain”, is prompted to confirm using data connectivity (the prompt might say “Connect to GhostLink Cloud to analyze? [Yes] [No]”), and if yes, behind the scenes the module enables the network, sends the query, and within a few seconds displays the answer. Afterwards, the module immediately disconnects or disables network to return to offline mode (the UI can indicate “Going offline...”). The entire transaction might take, say, 5–10 seconds depending on connection – which is acceptable given it’s a user-invoked action. Also, the response is stored in the Vault (e.g. appended to a `analysis_log.txt` with the code and answer), so that in the future if the same issue occurs, the device could recall the previous explanation even without internet. In essence, over time the Vault accumulates a **local knowledge base of troubleshooting Q&A** that improves the tool’s offline capabilities. This aligns with GhostLink’s idea of growing its knowledge through use, but in a deterministic, user-controlled way (nothing is sent or learned unless explicitly asked).

- **Error Handling and Robustness:** The OBD module is built to handle real-world conditions gracefully. If the vehicle is not connected or ignition is off, queries will timeout – we detect this and inform the user (“No response from ECU – please check connection and ignition.”). If the connection drops (vehicle turned off mid-session), the module catches communication errors and shows a notice (and perhaps auto-retries a few times). The module will also ensure that it doesn’t send requests too rapidly (especially on older K-Line cars) to avoid overwhelming the ECU. We’ll incorporate small delays or rely on the library’s internal pacing. Since the OBD port power might go off with ignition, if the Pi is being powered by OBD, we might lose power entirely – to mitigate data loss, we will have already logged any results to the Vault as they were read, and the Pi’s own UPS or supercapacitor (if any) would need to kick in. (If running on a separate battery, this is not an issue; the Pi stays on even if car off.)
- **Logging and Vault Integration:** Each OBD scan session can be logged. For example, when DTCs are read, we save the codes (and time, and maybe vehicle VIN) to a log file or database entry. This way the user has a history of issues encountered on each vehicle. If the user enters a vehicle identifier (or VIN is read), the log can group by vehicle. This is handy for tracking recurring problems. Live data streams could also be logged if the user engages a “Log data” mode (for instance, to record a driving session’s sensor data for later analysis on a PC). Initially, we will implement basic logging for codes and freeze frame snapshot only, to keep scope manageable. Those logs reside in the Vault and can be accessed or exported later.
- **User Experience Example:** The user plugs the GhostLink tool into their car’s OBD-II port. They select OBD mode on the screen. The device shows “Connecting...” and then “Connected (CAN 500k)” indicating it found the CAN bus at 500 kbps for example. The user then taps “Trouble Codes”. The screen lists: “P0301 – Cylinder 1 Misfire Detected” and “P0420 – Catalyst System Efficiency Below Threshold (Bank 1)”. The user taps P0301. A detail view shows the code description and maybe any freeze frame data (RPM, temp at fault time, etc.). The user then taps “Explain”. The device prompts for internet; they confirm. Within a few seconds, an explanation appears: “Misfire in cylinder 1. Likely causes: 1) Ignition (spark plug or coil), 2) Fuel injector, 3) Engine compression. Recommended to swap coil with cylinder 2 and see if misfire moves. If not, check spark plug condition...” etc. The user reads this advice. They then tap back. Perhaps they also tap P0420 and get an explanation for that (like possible O2 sensor or catalytic converter issues). Armed with this info, they go to the

mechanical part – maybe they use the Parts checklist to note “remove cylinder 1 coil and spark plug” as tasks, and use the torque spec reference to find spark plug torque. In this way, the OBD module has seamlessly integrated with the rest of the tool, and the user did not need a separate phone or manual to look up the codes – the GhostLink device itself became the guide, **offline except for the brief moment of fetching AI help**.

Overall, the OBD-II module transforms the GhostLink tool into a **full-fledged automotive scanner**, with capabilities on par with consumer OBD scanners plus the unique AI assistance edge. The design leverages known libraries and hardware interfaces to ensure reliability (e.g. using established ELM327 protocols ¹⁴ and interface boards proven in projects ¹). It remains **extensible** – in future, if we add support for other protocols (say CAN FD or Tesla’s API, etc.), we can extend this module or add new ones. But the first version will cover the 99% use-case: standard OBD-II in cars, with robust code reading and basic data monitoring.

Mechanical Logic Module (Parts & Torque Management)

The mechanical logic module is a unique aspect of GhostLink that goes beyond typical digital tools – it helps the user keep track of physical parts and specifications, acting as a **digital checklist and reference guide** during repairs or builds. Here’s how this module is structured and operates:

- **Part Tracking (Checklist) Implementation:** Under the hood, the module maintains a simple list (or array) of part entries. Each entry can contain fields like `name` (string), `status` (e.g. “removed”, “installed”, or a boolean for checked/unchecked), and maybe an optional `note`. When the user “adds” a part in the UI, the module either appends a new entry with the given name or toggles an existing one. For ease, we might pre-populate some common entries based on context. For instance, if an OBD code suggests an issue with Cylinder 1, the tool could automatically suggest adding “Cylinder 1 ignition coil” or “Cylinder 1 spark plug” to the checklist (this would be a smart integration between modules – possibly done via a macro or just the user manually adding). In general, though, the user will add items manually: the module provides an interface to do so (could be a function `add_part(name)` that the UI calls when the user inputs a new name).

The status toggling happens when the user taps the checkbox: the module flips the boolean and updates the display to checked (perhaps also appending “(done)” or strikethrough text). These changes are immediately saved to the Vault (for persistence). A simple approach: the parts list could be stored in a JSON file like:

```
"parts_session_2025-08-08": [  
  {"name": "Spark Plug #1", "status": "removed"},  
  {"name": "Ignition Coil #1", "status": "removed"}  
]
```

and as the user checks them off, update status to “installed” or “done”. If we want to keep it simpler, not differentiating removed vs installed but just whether addressed or not, we can use a boolean done flag. The UI then might separate “Open tasks” and “Completed tasks” for clarity. The module can allow multiple checklists (if user starts a new session or job, the old list could be saved with a timestamp and cleared for a fresh start). However, initially it might just maintain one active list at a time.

- **Torque Specification Storage:** The module holds a dictionary or database of torque values. For quick implementation, a JSON or YAML file in the vault (e.g. `torque_specs.json`) will map part names or bolt types to torque values and units. For example:

```
{
  "Wheel Lug Nut": {"value": 110, "unit": "Nm"},
  "Oil Drain Plug": {"value": 30, "unit": "Nm"},
  "Spark Plug": {"value": 20, "unit": "Nm"}
}
```

We can populate it with some generic defaults. The module provides functions like `get_torque(part_name)` which does a lookup (case-insensitive maybe) and returns the spec if found. If multiple items match or if a part has multiple fasteners with different specs, we might store an array or multiple entries (but that complicates UI; likely assume one key = one value for now). The UI's search bar essentially calls this function and displays the result. If not found, it shows "No spec found" and optionally allows the user to add one. Adding one means calling something like `set_torque(name, value, unit)` which updates the JSON and writes it to file. We will ensure that the values are stored in a standardized unit (Nm as default internally, and convert if user wants ft-lb in display). We might allow unit switching in settings as well, but since the user can input a value, maybe they specify the unit with it (e.g. a toggle or drop-down when adding). The module could keep everything in Nm internally for simplicity and just note if user prefers ft-lb for display, do the conversion on output.

- **Integration with OBD/AI:** Currently, the mechanical module is mostly user-driven (the user decides what parts to track and what to lookup). But we have opportunities to integrate it with data from other modules. For example, after scanning OBD codes or after the AI suggests "check spark plug and coil," the tool could prompt "Add these items to your part checklist?" – if user agrees, it automatically populates them in the list. This would be a nice quality-of-life feature bridging the diagnostic insight to action. It could be implemented via a macro or simply in the code when an AI response is received: parse out any part names and ask to add. This is a stretch goal for first version, but we mention it as a design consideration. Similarly, if a known procedure is started (perhaps in future packs, like a macro for "Replace Brake Pads"), the macro could load a predefined checklist of steps/parts (e.g. "Loosen lug nuts", "Jack up car", "Remove wheel", etc., with torque specs for lug nuts given). So the architecture is ready for such "recipe-based" checklists.

- **User Workflow:** A common use-case – the user is performing a repair or assembly:

- They navigate to Mechanical > Part Tracking. They tap "Add Part" and type "Brake Caliper Bolts". It appears on the list, unchecked (meaning to be done/removed). They add more: "Brake Pads (Left Front)", "Brake Pads (Right Front)", etc., creating a to-do list for their brake job. As they work, they check off each item: after removing the caliper bolts, they tap it – it becomes checked (or moves to a "removed" section). After installing new pads and putting back bolts, they mark them accordingly. This helps ensure all parts that came off go back on – no leftover screws on the bench! The list persistently stays even if they switch screens (they might go to OBD to check something mid-job; the list remains in memory or vault).

- When tightening the caliper bolts, they want the correct torque. They go to Mechanical > Torque Specs. They scroll or search for “Caliper” – suppose we had no entry by default. They have their service manual on hand which says 75 Nm. So they tap “Add Spec”, enter “Brake Caliper Bolt – 75 Nm”. Now it’s saved. Next time, it’s there. They then tighten to 75 Nm with their torque wrench. If we had an entry, it would have shown up directly. (In the future, one could imagine connecting a Bluetooth torque wrench and verifying it – but that’s beyond current scope.)
- Optionally, after finishing, the user could save or export the part checklist for record (the tool automatically saves it with a timestamp anyway in vault). They could mark the job as completed, clearing the active list (perhaps archiving it). For now, it might just keep accumulating items until manually cleared.
- **Data Persistence:** The parts list and torque entries are stored in the Vault as described. The parts list might be cleared on each new session unless we implement multiple named lists. Perhaps simplest: each time you go to part tracking, it continues from last session’s list unless user hits a “New Session” button. We’ll at least timestamp entries in the log so it’s clear when tasks were done. The torque spec file persists and only grows when new entries are added (we can also allow deletion of an entry via the UI if a mistake was made).
- **Scope and Limitations:** We keep the mechanical module intentionally basic in this first iteration. It’s essentially a glorified checklist and a note-taking tool for numbers. However, this is very powerful when combined with the rest of GhostLink: It turns our digital tool into not just a passive reader but an **interactive helper** in the repair workflow. It also cements GhostLink’s role as a *knowledge repository* – the torque values and notes you accumulate live on the device (so you don’t have to recall them or find the manual next time). This resonates with the GhostLink idea of a self-sufficient lab tool: e.g., “*GhostLink might integrate a Vault that can sync between your devices, providing access to documents and backups anywhere you go without relying on third-party servers.*”¹⁵. In our case, the Vault holds user-entered mechanical info which could be synced to another GhostLink device if needed (future mesh features).

To sum up, the Mechanical logic module adds a **human-centric logic layer** to the tool. It doesn’t measure or scan – it helps organize the *thought process and reference info* during hands-on work. By including it, GhostLink sets itself apart from single-purpose gadgets; it starts to become an **all-in-one garage assistant** that remembers both what the sensors say and what the user did in response.

Vault and Grant: Sovereign Data and Permission Control

Two of the defining features of GhostLink are **Vault** (the secure, offline data store) and **Grant** (the explicit permission system). We have woven these into the design of the diagnostic tool to ensure that user data remains under control and that any potentially sensitive operation is gated behind user consent. Here’s how Vault and Grant integrate into our application architecture:

- **Vault Integration (Data Sovereignty):** The Vault is the heart of GhostLink’s data storage. In this diagnostic tool, **all data and knowledge that the tool uses or produces is stored locally in the Vault**, never in some remote cloud. This includes:
- **Configuration & Knowledge Data:** As described, things like OBD-II code definitions, torque spec tables, and UI configuration live in vault files (JSON, vault text files, etc.). This means the tool carries

its knowledge with it – even with no internet, it knows, for example, what a given fault code means because that info is pre-loaded in the Vault. We have effectively “*embedded an expert knowledge base offline*” to the extent possible, fulfilling the sovereign, offline-first goal. If the user wants to update or expand that knowledge (say add more code definitions or reference docs), they can place files into the Vault. For example, a user could drop a PDF of their car’s repair manual into a `vault/docs/` folder – while the first version of our tool might not have a PDF viewer, it’s there and could be opened via a maintenance shell or a future feature. Similarly, if an AI query returns valuable info, we cache it in the Vault (e.g. store the Q&A text in a file) rather than relying on querying again. This is akin to creating a growing library of solutions over time on the device itself.

- **Operational Logs and State:** All usage data – sensor logs, part checklists, past diagnostic results – are saved in the Vault (or an associated local database). We do not send this telemetry anywhere else. The Vault can be thought of as the user’s personal database. This means the user can trust that their vehicle’s data or their notes are not being uploaded without their knowledge. It also means the device can function in a closed environment with no connectivity and still maintain full functionality and history.
- **Vault Security:** By default, the Vault data is plain (for transparency), but since the device is physically in the user’s possession, that is acceptable. If needed, encryption could be added (e.g. an option to encrypt certain vault files with a passphrase), but typically the threat model is that the user trusts themselves. However, GhostLink concept advocates for a “**zero-trust**” **internal stance** as well, meaning even modules shouldn’t freely access data they don’t need. In our design, modules access vault data relevant to them (the OBD module reads the DTC definitions file, but doesn’t need to read the part checklist, for instance). If we were to enforce this, we could implement simple access rules in code (or use Linux file permissions). The GhostLink research envisions each tool carrying a signed policy of what it can access ¹⁶, and requiring capabilities to access vault files not explicitly allowed ¹⁷. In this first version, we won’t implement a full capability system, but we respect the principle by segregating data logically per module and keeping the scope of operations limited. Nothing in our tool will ever attempt to access, say, the user’s personal files outside the ghostlink directory; it operates within its sandbox of known vault files.
- **Vault as Knowledge Base for AI:** One powerful aspect is using the Vault in conjunction with AI. As GhostLink’s approach suggests, “*instead of an AI model training on everything, use the Vault and project files as an external knowledge base,*” retrieving relevant snippets to assist the AI ¹⁸. We can utilize this in a limited way: for example, if the user asks a general question (maybe in a future update where a user can query “how do I do X?”), the system could search Vault documents (notes, saved web articles, etc.) for relevant info and feed it to an offline AI model. In our diagnostic scenario, we’re already doing a simpler version: feeding the AI the context of a fault code and perhaps referencing known common fixes (which could be stored in a vault file of known fixes) to get a tailored answer. As the device accumulates more data (like “previously, fixing P0301 required replacing coil”), it could even automatically include that in context. This is not fully implemented yet, but our architecture is **ready to leverage Vault data for smarter insights** down the line.

The net effect: the Vault makes the GhostLink tool *increasingly valuable over time* because it retains every piece of useful information learned or input. Unlike a typical scanner that forgets everything when turned off, GhostLink will “remember” (deterministically) your past diagnostics, forming a personal knowledge

vault. And crucially, all this data stays **local, private, and in your control** – aligning with the sovereign computing ethos that “you’re running your own open-source services on hardware you control” ¹⁹ .

- **Grant Integration (User Consent & Security):** The Grant system enforces that **nothing happens on the device without user permission**, especially actions that reach outside the device or could affect other systems. In our tool, this is manifested in several ways:
- **Network Access Control:** By default, the tool starts with networking off (or at least not actively communicating). When the user triggers an action that requires internet – e.g., the AI fault explanation – the system will **present a consent dialog** explicitly asking for permission to go online and send the relevant data. For example: *“Tool is requesting access to the network to fetch AI analysis. Allow once / Always allow / Deny?”* This dialog is very much in line with GhostLink’s plan for sovereign consent prompts ²⁰ . If the user taps “Allow once”, the tool brings up the Wi-Fi or uses a paired phone connection just for that transaction, then disconnects. “Always allow” could set a flag so that subsequent AI queries in the same session don’t prompt repeatedly (though for security, we might still ask each session). If “Deny”, the operation is canceled and no connection is made. The default state is deny (nothing goes out without a tap). Technically, this might be implemented by having the network interface down and only bringing it up in code when allowed. Or even if the device stays connected to Wi-Fi, we ensure no automatic requests are made – the only code that calls out is wrapped in a permission check. This addresses privacy (no accidental data leaks) and also aligns with *air-gapped mode* – a user could choose to never use the online feature, and the device would still function fully offline indefinitely.
- **Hardware/OS Actions:** Other potentially sensitive actions include modifying system state or performing any operation that could have side effects. For example, clearing OBD codes – one might consider that a potentially undesired action (since it wipes ECU info). We therefore include a user confirmation for that, as described (a yes/no prompt). Another might be firmware updates to the device or installing new packs – those would also require explicit user initiation (the device will never auto-install anything in the background). If we had a case where a tool module wants to access a file outside its area, we would prompt as well. Essentially, the software follows the principle: **“deny by default”** for any privilege not explicitly granted, similar to how smartphone apps ask for permissions ²¹ .
- **Isolation of Functions:** While on Linux not everything is capability-secured at the kernel level in our simple build, we logically isolate functions. The OBD module, for instance, will only communicate with the serial port and perhaps create a log file – it won’t, say, try to send data over network or read other vault files. If some breach or bug attempted to do so, in a future hardened version we’d have a microkernel or container preventing it. In this version, our safeguard is simpler: code discipline and maybe some user oversight (like an advanced user can review logs or an “activity monitor” UI showing what the system is doing). We also plan to implement an **airplane mode** (or “Air-Gapped Mode”) in settings that physically disables network interfaces until toggled. This ensures the device cannot connect out even if a malicious code tried, providing peace of mind. GhostLink’s vision includes *“air-gapped mode where it physically can’t connect out unless toggled on”* ²² , and we adhere to that concept.
- **Consent Logging:** Every time a Grant prompt is shown and the user responds, we can log that event (e.g. “User allowed network access for AI at 10:30PM” in a security log). This creates an audit trail for transparency – the user (or an auditor of the system) can later verify that nothing happened behind the scenes. It’s part of building trust that the system truly only does what it’s told. This log could be in the Vault as well (maybe in a `grants.log` file).

- **Example of Grant in Action:** The user hits the “Explain code” button. The system pauses the UI to show a dialog: *“GhostLink Diagnostic wants to enable Wi-Fi and send the code P0420 to GhostLink AI for analysis. This will use a brief internet connection. Allow this?”* with two choices: *Allow* or *Cancel*. The user taps *Allow*. Immediately, the network manager is invoked to connect (if not already connected; perhaps the user had pre-configured a Wi-Fi network, or else we might even tether via phone Bluetooth – but assume Wi-Fi available). The request is then sent to the AI service. Once a response is received and displayed, the tool brings down the Wi-Fi (or at least it will not send any further data). If the user had tapped *Cancel*, the dialog would close and maybe a message “Operation canceled – no internet used” is shown. The system default is always to not proceed without explicit confirmation. Another example: Suppose in a future iteration we have a feature to upload logs to a personal server or share data with a mechanic. That too would trigger a Grant dialog summarizing what data would be sent and to where, requiring yes/no. By making these interactions explicit, we reinforce the user’s sovereignty over the device’s actions. Even though this is a single-user device, we treat it as if it were a multi-tenant environment where **each tool must ask the “owner” for permission** for significant actions. This layered consent is what GhostLink calls *“fortress of intent enforcement”*, ensuring even if some part of the system were compromised, it can’t silently exfiltrate data or cause havoc because it doesn’t have the capability granted ²¹.
- **Grant and System Updates:** One special case is updating the tool or adding new packs. Our plan for update delivery (discussed next) will also be governed by Grant. For instance, if the device detects a USB drive inserted with a new pack file, it will ask the user if they want to install it. If the device goes online to check for updates, that check itself is only done on user command (or with prior permission) – no phoning home automatically. This ties into both security and user control: updates can be a vector for supply chain attacks, so GhostLink will likely implement signed packages and require user approval to apply them.

In conclusion, Vault and Grant together enforce that the GhostLink Diagnostic Tool is **user-centric** in data and control. The Vault means all valuable data is **yours and local** (and can be backed up or migrated by you as you wish), and the Grant system means the device will **never violate your intent or privacy** by doing something covertly. These features instantiate GhostLink’s foundational principles (sovereign data, zero-trust posture) in practical terms within our diagnostics application. They give the user confidence that even though the tool has powerful capabilities (like internet access or automated reasoning), those capabilities remain *fully under user command*. This builds trust in the tool as an extension of the user, not an agent of some cloud service.

Power Management and Efficiency

Since the GhostLink tool is meant to be portable and possibly battery-powered, we’ve included strategies for power handling and energy efficiency. Additionally, we consider safe shutdown and resiliency under varying power conditions (vehicle environments can be harsh on electronics). Key points of power management in our design:

- **Flexible Power Input:** As noted, the device can run off a USB power bank or the car’s own power (via OBD port or cigarette lighter). In automotive use, power can be noisy (voltage spikes, drops). Our hardware design includes power conditioning – the OBD interface’s DC-DC regulator will handle automotive transients (typically those OBD boards have protection for load dump, etc.). If on a generic USB bank, the power is cleaner but limited in capacity. We assume a typical use-case of a few

hours on a decent power bank is feasible, given the Pi 5 consumption and the peripherals. The Pi 5 might draw around 600-1000 mA when busy with the display on. We will be mindful to reduce needless power draw to extend runtime.

- **Dynamic Power Saving:** In software, we can make the Pi operate more efficiently:

- We will **disable Wi-Fi and Bluetooth radios** when not needed (the Pi has `rkill` or `ifconfig down` we can use). Wireless modules can draw a significant idle current, so turning them off except during the brief AI query is beneficial.
- The display backlight can be turned down or even off when idle. For example, if the user hasn't touched the screen for 5 minutes, we could dim the backlight to a lower level (with a subtle reminder on screen like "Sleeping – tap to wake"). This is like a screensaver. The user can configure the timeout. This helps if the device is left on while not actively being looked at. Similarly, if the CPU is idle (no active scanning), the Pi's CPU frequency scaling will naturally throttle down to lower frequency to save power – we ensure that the default governor is `ondemand` or `powersave` when appropriate. The GhostLink app, being mostly event-driven (waiting for user input or periodic sensor readings), often will not tax the CPU heavily (except maybe during AI request which is short).
- If using the device as an always-on monitor in a vehicle (less likely, but possibly to monitor sensors while driving), we might not want to suspend, but we would still benefit from turning off unused features and efficient coding (we'll ensure loops have sleep delays rather than hogging 100% CPU polling).
- **Thermal Considerations:** The Pi 5 can get warm under load. We might include a small heatsink or ensure the case is vented. Lower power usage also means less heat, which is good for stability in hot garages or underhood testing. If the device overheats, it could throttle – our software should avoid pushing it into that region (only scenario that might is if we had a heavy local AI process, which we currently offload to the cloud instead).
- **Battery Level Awareness:** If running on a USB battery, it's not straightforward to get the battery level unless the power bank has some interface (most don't). We might approximate by measuring the Pi supply voltage via an ADC channel (some UPS HATs provide this). Without extra hardware, we cannot directly know the charge. Instead, we rely on user observation or an external battery's indicators. If using vehicle power, we *can* infer some things: the ADC can measure the car's system voltage (if we connect a voltage sense to OBD 12V through a divider). This could tell us if the engine is off (voltage ~12.0) or on (~14.4). If the engine is off and voltage is dropping, it might indicate the tool is draining the car's battery – though our tool's consumption is relatively low compared to a car battery capacity for short sessions. Nonetheless, we might implement a precaution: if the device is powered by OBD and it detects battery voltage falling below a threshold (like <11.8 V), it could warn the user that the car battery is low and maybe suggest shutting down to avoid draining it. This is more of a future enhancement, but the pieces are there (we have ADC and can read that pin if wired).
- **Graceful Shutdown:** A critical aspect of power handling is shutting down safely to avoid file corruption. We provide a **Shutdown option in the UI** (in Settings menu). When invoked, it will exit the app, call a system shutdown (which syncs disks and halts the OS). The user can then remove power. If the device is powered from the car and the car is turned off (cutting power suddenly), we

want to mitigate damage: ideally, we'd have a small backup power source (some designs use a supercapacitor or a small backup battery) that gives the Pi a few seconds to shut down. This might be beyond the current scope, but we'll note it. Alternatively, if the Pi is running on its own battery, sudden removal isn't an issue (the Pi will run until battery dies, and hopefully the user uses the UI to shut down before that). We might add a feature where if the Pi detects the input voltage from OBD dropped (car shut off), it automatically initiates shutdown after, say, 1 minute, unless external power returns. This requires that voltage sensing mentioned above. Many automotive Raspberry Pi projects address this with a dedicated power management HAT – we could incorporate one in future (for now, we mostly assume the user will manually shut down or keep it powered by a stable source).

- **Efficiency in Code (No Bloat):** True to GhostLink's principle, we keep background processes to a minimum, which inherently saves power. For example, not running a full desktop means we're not wasting CPU/GPU drawing unnecessary UI elements or running update checkers. No cloud sync means no network chip activity draining power. The only significant usage is the actual tasks the user asks for (which is how it should be). In effect, "*GhostLink's design of no background processes or self-modifying code*" ²³ is as much about predictability as it is about efficiency. We avoid the typical drift where an idling system uses more and more CPU over time – our system at idle should settle to a very low utilization (maybe a few percent CPU for the UI loop and periodic sensor checks). This could allow the device to run for many hours on battery if mostly idle.
- **Power Usage Scenario:** Suppose the user is using a 10,000 mAh USB battery (~ at 5V, that's 50 Wh). The Pi 5 with screen on might draw ~5 W average during active use. That battery could theoretically run it ~10 hours. With our optimizations (screen dimming when possible, Wi-Fi off except brief moments), maybe we shave that to 4 W or less, extending use. If doing heavy AI (which we don't locally) or leaving Wi-Fi on streaming data, it would be higher. But our typical use – measure voltage, read codes, etc. – is sporadic and low-intensity. This is acceptable for a day's work on a car. In a vehicle, if plugged in, the car's battery (when engine on) easily supplies it, and when engine off, as long as usage is moderate, it shouldn't kill the car battery (especially if our tool auto-shuts down after a while off).
- **Wake/Sleep (Future):** We don't have a true sleep mode because the Pi is not great at deep sleep. However, one could implement a pseudo-sleep: turn off display and put CPU in low frequency, effectively pausing operations. We might provide a "Suspend" button (which just blanks the screen and perhaps stops polling loops) as a way to conserve power if the user is stepping away. The user then taps the screen or power button to resume. This would conserve some energy. A more advanced approach could be to utilize the Pi's halt state with a wake on GPIO (if any hardware allows) – possibly overkill now.

In essence, our power management philosophy is **efficiency-by-default**. The device only uses significant power when it's actively doing work for the user; otherwise, it stays cool and sipping minimal energy. This ensures that being "*battery-agnostic*" truly means the user can hook up whatever power is at hand and not worry that the tool will rapidly drain it. When plugged into the grid or car, it uses what it needs but also avoids unnecessary stress on the car's electrical system. All critical operations degrade gracefully with power changes (no data loss on sudden off, warnings on low power, etc.). This reliability in power handling is crucial for an embedded tool – nothing is worse than a diagnostic tool that shuts off mid-operation or corrupts its system due to power issues. We've taken measures in design to prevent that.

Update Mechanism and Future Expansion Packs

One of the core requirements is that this first GhostLink tool serves as the foundation for future **modular expansion packs** (like a Fabrication Pack, etc.). Therefore, we have designed an update and extension mechanism that allows new features and hardware to be integrated **without replacing the whole system**, maintaining the GhostLink toolbox concept. Here's how we approach updates and add-ons:

- **Modular Architecture for Packs:** As mentioned, our file structure and program logic anticipate additional **"packs"**. A pack in GhostLink terms would include new tool modules, vault entries (macros, data), UI components, and any necessary drivers for new hardware. For example, a **Fabrication Pack** might add a CNC control module and macros for interpreting CAD files, plus maybe require connecting an Arduino-based CNC controller to the Pi. Our system can handle this by:
 - Having a `packs/` **directory** where each pack is a subfolder. E.g., `packs/fabrication/` could contain `fabrication.py`, its vault extensions (maybe `fabrication.vault` with macros or data), and any resource files (like G-code templates or such). The main program on startup scans `packs/` and dynamically imports any modules it finds, adding them to the menu. We can use a simple manifest file in each pack (like `pack_info.json` describing the pack's name, version, and which menu or commands to add). This way, adding a pack is as easy as dropping its folder onto the device. This approach aligns with GhostLink's notion of being composed of *"small building blocks out of which sophisticated systems can be composed"* ²⁴. Each pack is a building block that plugs in.
- Ensuring **isolation** between packs: While packs can inter-communicate through the main program or defined APIs (for example, an AI pack might enhance the capabilities of another pack's functions), they should not arbitrarily interfere. The Vault can have separate sections for each pack's data (or at least name-spaced entries). Our permission system can extend to packs – e.g., if a new pack wants to access the network or certain hardware, it would invoke the same Grant prompts appropriate to that action (just as our base modules do). This keeps new additions from breaking the security model.
- **Delivery of Updates/Packs:** We plan for both **offline and online update methods**:
 - **Offline (Sneaker-net) Updates:** Since GhostLink is offline-first, a common way to add a pack would be manually. The user could download the pack file (for instance, a `.zip` or `.tar.gz` containing the pack folder) from a trusted source (maybe on another computer or provided on a USB drive) and then copy it to the GhostLink device. If the device has USB ports or a method to copy files (like mounting a USB drive or using SCP over the local network), the user can place the pack into a designated directory (like `/opt/ghostlink/packs/`). The GhostLink software could have an "Import Pack" function in the UI that scans an inserted USB for known pack files and copies them in place (with confirmation). Once the pack is in the packs directory, on next boot (or immediately via a refresh command) the system detects it and enables its features. We will ensure that the base system is tolerant of unknown/new packs – e.g., if a pack requires a certain newer core, the pack manifest might specify a minimum version so the system can warn the user if incompatible.
 - **Online Updates:** Optionally, for users who allow internet connectivity, we can provide an online update mechanism. This would involve the device checking a GhostLink repository for updates. Importantly, this check would only run when the user initiates it or has explicitly allowed auto-checking (we may include a "Check for updates" button in Settings, which when pressed will do a quick online query – after a Grant confirmation, since it's network use). If a new core version or pack is available, it can list them and ask the user if they want to download and install. The installation

process would then fetch the pack over HTTPS, verify its integrity (with a cryptographic signature to ensure it's an official release), and then place the files in the right spot. The UI would show progress and prompt a reboot or reload of the app if needed. For small packs, we might even allow hot-loading without a full reboot (just integrating the new menu on the fly).

- In both methods, **security is paramount**: Packs should be signed by GhostLink developers (or the user should inspect them if third-party). We could use a public/private key scheme where the device has the public key in its trust store, and any pack or update must be signed. This prevents tampering or unauthorized code from being loaded. In practice, maybe the first version will not have a full PKI, but we strongly consider it given the sovereign stance (the user might be the only signer if they build their own packs). Regardless, the update mechanism won't run arbitrary code from unknown sources without user consent.
- **Update of Core System**: Aside from packs that add features, occasionally the **core GhostLink OS or core tool** might need updates (for bug fixes, improvements). Since we want a deterministic system, we likely won't do self-updates silently. Instead, similar to packs, the user would get an update package for the core. This might be delivered as a "Release Pack" (we have a template for release pack in user files, indicating a process). The user can apply it via the same methods. Possibly, core updates might require reflashing an SD card if it's an OS-level change – but we'd try to avoid that by separating the app layer from the OS. If our app and packs sit on a data partition, we could update them without touching the base OS image. Indeed, using something like an immutable OS (the OS stays fixed, we just update the userland apps) is a good approach. But for now, a simple method is fine: e.g., an update script that replaces the old ghostlink modules with new ones from an update pack. Always, we do it in a fail-safe way (keep a backup of old version until new one is verified running).
- **Future Hardware Expansion**: When new hardware modules are introduced (say the Fabrication Pack comes with a specific PCB to control machinery), the software can include the necessary drivers or libraries. For example, if it's a 3D printer control, the pack might include a Python library for G-code interpretation or require enabling the serial port for an Arduino. Our update mechanism can handle this: the pack's manifest could specify if any system config changes are needed (like "enable I2C" or "install this Debian package"). We can either automate those changes or provide instructions to the user. Because our user base might be tech-savvy (GhostLink appeals to DIY engineers), a manual step isn't the end of the world, but we aim to automate where possible. Possibly, we will ship major hardware expansions with an updated SD card image that has everything configured (especially if something like real-time kernel or special drivers are needed). However, ideally one could just plug the new hardware into the Pi, add the pack files, reboot, and it works – **plug-and-play modularity**.
- **Backward Compatibility**: The first GhostLink tool is the blueprint; as we add packs, we ensure they don't break existing functionality. That's why isolation and clear interfaces are important. For instance, if the Fabrication Pack adds a heavy dependency that could slow the system, it should remain dormant unless invoked. The core will remain lean. If the core itself evolves (say we switch UI toolkit or major version bump), we'd coordinate packs to match that version (the manifest version checking covers that).

- **Example – Adding the Fabrication Pack:** Let's illustrate how the future might work: GhostLink Fabrication Pack is released as `fabrication_pack.zip`. The user obtains it and wants to add it to their GhostLink Diagnostics tool. They copy the zip to a USB drive and plug it into the Pi (perhaps the Pi auto-mounts it or the user can use a provided file manager). In the GhostLink UI, under Settings, they choose "Install Pack" – the system scans the drive, finds `fabrication_pack.zip`, and verifies its signature. It then unzips it into `/opt/ghostlink/packs/fabrication/`. It sees in `pack_info.json` that this pack adds a new tool category "Fabrication" with description "CNC & 3D Printing tools". The system registers this and perhaps needs a reboot to load new drivers (if, say, a GPIO needs configuring for this pack). After reboot, the main menu now shows a new option "Fabrication Tools". Tapping that opens the UI defined by the fabrication module – perhaps controlling a connected device (the hardware for this pack might be a special HAT on the Pi; the module will detect it or prompt to attach it if not found). None of the original Diagnostics functions are affected; they're still there as before. The Vault might have new vault files from the pack (like macros for fabrication tasks), which are simply added to the vault directory. The user can now seamlessly switch between using diagnostic functions and fabrication functions on the same device, using the same GhostLink UI paradigm. This showcases the **scalability** of the GhostLink ecosystem: one device, many tools, all integrated. Importantly, each pack (including this first diagnostics one) can be developed somewhat independently thanks to the modular architecture – they share the core runtime but have their own logic.
- **Maintenance and Testing:** With each new pack or update, we will have a **test plan** to ensure it doesn't break existing features (the user files included a `testplan_pack.md` template, suggesting we do test each feature and pack thoroughly). For example, after installing Pack X, we'd run through a suite of diagnostic tests to ensure voltage reading still works, OBD still connects, etc., as well as tests for the new functionality. This discipline will maintain reliability as the system grows.
- **Delivery Strategy:** Initially, since GhostLink might be a personal project at first, distribution might be manual (GitHub releases, etc.). Eventually, if it's meant for broader use, we could have an update server or repository (maybe decentralized, given the ethos). But from a design perspective, we allow both manual and automatic means as described, with user consent always in the loop.

In summary, the update and expansion mechanism ensures that this "cold version" of the GhostLink tool can **evolve without losing its core integrity**. The first diagnostics tool is simple and self-contained, **suitable for bundling or direct use** as requested – for instance, we could ship it as an SD card image that someone can just plug into a Pi and have this functionality out of the box. From that base, users can customize and expand according to their needs: if they want only offline diagnostics, they can stick to this version indefinitely; if they want new features, they can selectively add packs. This approach positions GhostLink as not just a one-off gadget, but an **ecosystem** that can gradually replace more and more of traditional tools and software, all under a unified architecture. We've thus designed the first tool not in isolation, but as the **ground floor of a much larger vision** – with the care that each additional story will rest securely on this foundation.

Build and Deployment Steps (Detailed Guide)

Finally, to tie all the above design into a practical outcome, here are the **step-by-step build instructions** and deployment notes for assembling the GhostLink Diagnostic Tool:

1. Hardware Assembly:

- **Prepare the Raspberry Pi:** Use the Raspberry Pi 5 (if available) or Pi 4 Model B with at least 2GB RAM (for comfortable UI operation and any AI needs). Attach heatsinks if provided (for cooling).

- **Connect the Touchscreen:** Mount the official 7" Raspberry Pi touchscreen or your chosen display. For the official one, connect the DSI ribbon cable to the Pi's display port and attach the USB for touch (if required, some connect via GPIO for touch). Secure the Pi to the display's adapter board as per its instructions. If using an HDMI touch display, plug it into HDMI and connect USB for touch interface. Test that the screen powers on.

- **Attach OBD-II Interface:** If using a Pi CAN/OBD HAT (e.g., PiCAN2 or a custom board), seat it onto the Pi's GPIO header. Connect the OBD-II cable to that board's DB9 or OBD connector. If using a separate OBD module like an ELM327-based device (USB or serial), connect it: for a USB OBD adapter, plug into Pi's USB; for a serial OBD module, wire its TX, RX, Ground, and 5V appropriately to the Pi's GPIO (TX of module to RX of Pi GPIO14, RX to TX GPIO15, etc., plus power and ground). Also connect the module's K-line/L-line if needed to the OBD port – if using a ready-made board, this is handled. Ensure the interface's power is sorted (some HATs will take 12V from OBD and power the Pi; others you power Pi via USB and it powers the OBD interface at 5V). Double-check jumpers/settings on the interface for correct CAN termination, etc., as per its documentation.

- **Wire the ADC (Analog Module):** Connect the MCP3008/MCP3204 ADC to the Pi. For MCP3xxx series:

- VDD and VREF of ADC to 3.3V (Pin 1 and 17 on Pi for 3.3V).

- AGND and DGND of ADC to Pi Ground (any ground pin).

- ADC CLK to Pi SPI SCLK (GPIO11, physical pin 23).

- ADC DOUT to Pi MISO (GPIO9, pin 21).

- ADC DIN to Pi MOSI (GPIO10, pin 19).

- ADC CS (chip select) to Pi CE0 (GPIO8, pin 24) – or CE1 if you prefer, just adjust software.

- Now, create the voltage divider input: e.g., a resistor from the probe positive to ADC channel 0, and another from ADC channel 0 to ground, to scale voltage. For example, use 100k Ω (from probe to ADC) and 20k Ω (ADC to ground) for roughly a 6:1 division (this allows reading up to ~20V as ~3.3V). Connect the probe positive lead to the top of this divider (and to no other connection), connect the midpoint to ADC CH0, and bottom to ground. The probe ground lead goes to Pi ground. Optionally, put a 5.1V zener diode across the input (cathode at ADC input, anode at ground) to clamp any high voltage spikes, and a small capacitor (0.1 μ F) for noise filtering.

- For continuity, designate another ADC channel or a GPIO: simplest is a GPIO method. Take a GPIO (say GPIO 5) and set it as output HIGH. Put a series resistor (~1k Ω) from that pin to the continuity test probe positive. The continuity test probe negative goes to ground (could be the same ground lead as the voltmeter – essentially they share the common ground). Then use another GPIO (say GPIO 6) as input with an internal pull-up disabled (we'll rely on the resistor from the other pin). When the probes are connected together or through a low resistance, GPIO 6 will read LOW (because GPIO5 via resistor drives the circuit and the current flows to ground). If not connected, GPIO6 might float – better to also put a weak pull-up on GPIO6 to define open as HIGH. Alternatively, we could use the ADC channel we already have: measure voltage on that probe – near 0 means continuity. But using a digital input is simpler for threshold detection.

- Connect the buzzer: a small piezo buzzer's one lead to a GPIO (say GPIO 12) and the other lead to ground (if the buzzer isn't polarized, either way; if it is, typically positive to GPIO, negative to ground). We might need a transistor driver if the buzzer draws more current than a GPIO can supply, but a tiny piezo usually is fine. You may also add a resistor (~100 Ω) in series to limit current. The buzzer will be driven by toggling the GPIO in software (or using PWM tone).

- **General Purpose Connections:** Make sure all grounds (OBD interface ground, ADC ground, Pi ground, etc.) are common – this is critical for proper measurements and communications. Tie them together at one point to avoid ground loops.

- **Powering Up:** Decide how to power during development – likely via Pi's USB-C power input from a wall adapter for now. Later, if testing in-car, you can power via the OBD (just ensure the interface is properly providing 5V to Pi or use the car USB adapter). Having a separate power switch or a smart UPS hat can be useful; at minimum, a way to safely cut power when needed. Now, hardware assembly is done. Double-check connections with a diagram to avoid mis-wiring (particularly ensure 3.3V lines and that the Pi doesn't get 5V on a GPIO by accident).

2. System Installation:

- **Prepare OS:** Flash a microSD card with Raspberry Pi OS Lite (64-bit) or a similar minimal Linux. You can use Raspberry Pi Imager or `dd`. Since we might incorporate custom configs, using Pi OS Lite (Bullseye or newer) is a good start. After flashing, enable SSH (by placing an empty `ssh` file in `/boot`, optional if you want remote access) and configure Wi-Fi if needed (add `wpa_supplicant.conf` in `/boot` with credentials, though we won't auto-connect by default for security – this is mainly for ease of headless setup).

- **Boot and Setup OS:** Insert the SD in Pi, connect keyboard/monitor for initial setup (or SSH in). On first boot, expand filesystem if not automatic. Create a user (if it's the new Pi OS, you'll be prompted; otherwise use default `pi`). Since this device is single-purpose, you can auto-login the `pi` user on console: edit `/etc/systemd/system/getty@tty1.service.d/autologin.conf` to autologin (or use `raspi-config` tool to enable autologin to console). Also, run `raspi-config` to enable SPI interface (for ADC) and enable Serial port for our use: *disable serial console* (since we need the UART for OBD) but *enable serial port hardware*. This typically means in `raspi-config` > Interface Options > Serial: say No to console, Yes to hardware port. This ensures `/dev/serial0` (or `ttyAMA0/ttyS0` depending) is free for OBD use. Also enable I2C if your ADC was I2C (not in our case), and enable PWM audio if using PWM buzzer (though usually direct GPIO toggling is fine).

- **Install Dependencies:** The GhostLink tool can be written in a high-level language like Python for rapid dev (or C++ for performance, but Python is fine here). Assuming Python: install Python3 and needed libraries. For example, `sudo apt-get install python3-pip python3-spidev python3-rpi.gpio` etc. Use pip to install `obd` library: `pip3 install obd` (this is the python-OBD module we discussed, which supports ELM327 adapters ¹⁴). Also install any GUI library – we have options: - If using a simple text/curses UI, ensure the console font is large enough. We might not need extra packages for that. - If using a lightweight GUI, perhaps PyQt or Kivy. For PyQt5, `sudo apt-get install python3-pyqt5`. For Kivy, it's a bit more involved (need SDL2, etc.). Another approach: Pygame for simple graphics – `sudo apt-get install python3-pygame`. Considering time, a curses or Pygame text-based fullscreen interface might suffice (curses for purely text; pygame to draw text with better control and capture touch events via SDL). - For now, let's assume either a text mode UI (which runs on the framebuffer console) or using Pygame. If we use Pygame, we might also need to set environment variables to use the SDL framebuffer driver (`export SDL_VIDEODRIVER=fbcon` and `SDL_FBDEV=/dev/fb0` and maybe input env for touchscreen). Make sure the touchscreen events are accessible (they usually appear as `/dev/input/eventX` – we may need to calibrate but the official touchscreen should map 1:1). - **Set Up Auto-Launch:** Create a systemd service for GhostLink: - Create `/etc/systemd/system/ghostlink.service` with contents:

```
[Unit]
Description=GhostLink Diagnostics
After=network.target
(maybe network.target if we want network up, but we might not; at least basic
system targets)
[Service]
```

```
Type=simple
User=pi (or root if needed for GPIO direct access; but better add pi to gpio,
i2c, spi groups)
WorkingDirectory=/opt/ghostlink (assuming we place code there)
ExecStart=/usr/bin/python3 /opt/ghostlink/ghostlink_main.py
Restart=on-failure
[Install]
WantedBy=multi-user.target
```

- Enable the service: `sudo systemctl enable ghostlink.service`. This will cause it to run on boot. - Alternatively, you can use the autologin method: add a line to `~/.bashrc` like `exec python3 /opt/ghostlink/ghostlink_main.py` so when pi user logs in on tty1 it launches the program. But systemd approach is cleaner and allows auto-restart if it crashes. - **Deploy GhostLink Code:** Copy the code files (developed on PC or directly on Pi) into `/opt/ghostlink/` (you may need to create this dir). This includes: `ghostlink_main.py`, the `modules/` folder with `electrical.py`, `obd.py`, etc., the `ui/` folder if any, and the `vault/` folder with initial data. Set appropriate permissions (maybe chown to pi:pi, and make the main script executable if it's a script). If using compiled binaries, place them accordingly and adjust `ExecStart` in service to that binary path. - **Configure Vault Data:** Place your JSON files for DTC descriptions and any starting torque spec or macros into `vault/`. For example, create `vault/dtc.json` with standard codes. Ensure the code can find these (the path might be relative to working directory or an environment variable like `GHOSTLINK_VAULT`). Possibly define an env in the service with `Environment="VAULT_DIR=/opt/ghostlink/vault"`. The first time run, the tool might also initialize empty logs or memory files in vault if they don't exist. - **Final OS Tweaks:** It's good to reduce OS writes if running on SD. Consider setting `Filesystem readonly`: not mandatory now but recommended if device might be power-cut. If not going read-only, at least disable swap (`dphys-swapfile swapoff`). Update `/boot/config.txt` to rotate screen if needed or set correct resolution for display. Also, add `dtoverlay=disable-wifi` and `dtoverlay=disable-bt` in config.txt if you want Wi-Fi/Bluetooth totally off by default (you can re-enable via command if needed, or comment those lines when planning to use network). However, we might keep Wi-Fi on but not connected. It's up to user preference; we can programmatically `rkill` block/unblock wifi.

3. Software Development & Testing:

- **Electrical Module Test:** Write a small test to read ADC values. For example, in Python, use `spidev`:

```
import spidev
spi = spidev.SpiDev()
spi.open(0,0) # bus 0, CS0
spi.max_speed_hz = 1350000
# MCP3008 read: send 3 bytes: start bit, channel config, dummy
adc = spi.xfer2([0x01, 0x80, 0x00]) # channel 0
raw_value = ((adc[1]&3) << 8) | adc[2]
voltage = raw_value * (3.3/1023) * divider_scale
```

Check that when you apply a known voltage (e.g. 3.3V, or use a potentiometer as in the instructables), the reading matches expected. Adjust `divider_scale` (the inverse of division ratio) accordingly. Also test

continuity: set the GPIO output high, read the input GPIO, short the probes – does the input read low? Use RPi.GPIO or peripheral library to test. Calibrate threshold if needed. Make sure the buzzer beeps when you set its GPIO high/low (some buzzers need PWM frequency ~2kHz; you can simulate by toggling or using Pi's hardware PWM via RPi.GPIO PWM function at a tone frequency). - **OBD Module Test:** Without a car, you can test basic connectivity. If you have an ELM327 adapter, connect a loopback or use an OBD simulator (there are PC programs or Arduino-based ECU simulators). Alternatively, test AT commands: `echo "ATI" > /dev/serial0` and see if you get a response from the ELM chip. The python-OBD library's `obd.OBD()` can be tried if an adapter is present. If no car, skip detailed tests or use known working environment (maybe test on a real car when possible). At least ensure that the serial port opens and that our code for reading DTC (if we send a Mode01/03 request) handles timeouts gracefully when no ECU responds. - **UI Test:** Ensure the UI launches on the touchscreen. You might initially test on HDMI with a keyboard for input. If using curses, just run it in the console. If using Pygame/SDL on console, you might need to set env variables and run as root to access /dev/fb0. Try drawing something on screen and ensure touch input produces events (PyGame should give MOUSEBUTTONDOWN events from the touchscreen if configured). Calibrate if necessary (xinput calibration if in X, or libinput if not – hopefully not needed if it's mapped 1:1). - **Integration Test:** Run the `ghostlink_main.py` outside of systemd first (just from the console) to see debug output and catch errors. Go through each menu: the Electrical should display a number (even if no input, it might show 0 or noise – test by touching the probe to 3.3V or ground to see it change). The continuity mode should reflect open vs short (maybe print "Open" then short the probes, see if it prints "Closed" and beeps). The OBD menu likely will try to connect – without a car, it might timeout, which is fine as long as it doesn't crash. If you have a known OBD code (some adapters can echo, or you manually craft a response), you could simulate one to test the parsing. The mechanical section: add a couple dummy parts, check/uncheck them, and exit – then inspect the vault file to see if it saved. Add a torque value and see if retrieval works. - **Permission Mechanism Test:** For network, it might be tricky to test if you're offline. If Wi-Fi is configured, trigger an AI query (which you might simulate by just hitting that code path and perhaps have it call a dummy function that prints "(Would call AI here)"). Confirm that the Grant prompt appears. If possible, temporarily allow internet and test a small OpenAI API call (with a valid API key in config). This will verify that enabling Wi-Fi and the HTTP request works. Then ensure Wi-Fi turns off after. If no internet, just ensure the logic flows correctly up to the point of making the request (which would fail, but the fail is handled). - **Systemd Integration:** Once tests are good, start the service via `sudo systemctl start ghostlink`. It should launch on boot. Reboot the Pi with everything connected (especially the display) and watch it come up into the app. If the screen goes blank then shows the UI, success. If something goes wrong, connect via SSH or serial console to debug (maybe the service output via `journalctl -u ghostlink`). Common issues could be permissions (e.g. need to add `User=pi` to groups: `sudo usermod -aG dialout,spi,gpio,i2c pi` so that pi user can access serial, SPI, etc. Alternatively run service as root, but we try to avoid root for safety). - **Auto-Start & Kiosk Behavior:** Verify that on boot it doesn't stop at login, and that exiting the app (if you add a secret exit key like F10) doesn't drop to a shell visible to user (unless you want that for emergency). Possibly disable Ctrl+Alt+F1 switching if you want to lock it down completely. But early on, not critical.

4. Documentation & Usage:

- Document the pin connections and any calibration constants in a README (for anyone replicating). Also note how to safely shut down (via menu).
- Prepare a **user guide** explaining how to use each function. This ensures the tool is understandable to others. It might include examples of interpreting OBD results or using the part tracker.
- Emphasize safety notes: e.g. not to connect to high voltage, not to use AI suggestions blindly for critical repairs, etc.

5. (Optional) Hardening and Packaging:

- If this is to be **bundled as a direct-use image**, after everything works, you can create an image of the SD card as the “GhostLink Diagnostics v1.0” release. This image can be flashed by anyone to identical hardware to reproduce the tool. That’s the simplest distribution. - If packaging as a software bundle, create a `.deb` or installer script that can turn a fresh Pi OS into this system by installing files and configs. But likely an image is easier initially. - Consider applying OS hardening: disable unused TTYS, enable firewall (though if no network, minimal risk), lock down sudo (maybe not needed if device is single-user). Also ensure the system clock is handled (if no internet, Pi will have wrong time on boot; consider adding an RTC or at least let user set time for accurate logging). - Test in realistic conditions: for instance, take the device to a vehicle, power from the car’s 12V via OBD or lighter, and try reading codes from an actual car’s ECU. Validate all works as expected in the field (sometimes noise or different ground potentials can cause issues – e.g., ensure the ADC ground is same as car ground through OBD when measuring car circuits). - Finally, incorporate any fixes from field testing and finalize the build.

By following these build steps, one should be able to go from raw components to a functioning GhostLink diagnostic unit. The end result is a **single, self-contained tool** – plug it into a vehicle or a circuit, and you have an all-in-one diagnostic aid (multimeter + OBD scanner + notebook), with the potential to grow further. With detailed architecture and careful construction as above, we’ve ensured this first GhostLink tool is not only *useful on day one* but also *future-proof* to carry the weight of upcoming enhancements in the GhostLink ecosystem.

Sources: The design and implementation choices referenced above are informed by a variety of sources and best practices. For instance, the need for an external ADC on Raspberry Pi is documented (the Pi “*has nearly all features... However the only drawback... lack of a dedicated ADC... solution is to use a dedicated ADC IC*” ²). The approach to OBD-II integration draws on existing Raspberry Pi OBD projects, such as the Elektor magazine project which successfully “*converts a Raspberry Pi into an OBD2 diagnostic tool*” with an add-on board ¹ and the use of the Python-OBD library which “*works with ELM327 adapters, fit for the Raspberry Pi*” ²⁵. The UI philosophy follows minimalist design principles (Nielsen’s heuristic that “*Interfaces should not contain information which is irrelevant... every extra unit competes with relevant info*” ¹¹). The Vault and Grant concepts are built per GhostLink’s sovereign computing model, where explicit user permission (consent dialog) is required for actions like network access ²⁰, and local vault files store state and logs (e.g. `sensor_trace.vault` logging hardware readings ⁶). Lastly, the plan for packs references how GhostLink packs can differ mainly in content (macros, data) loaded on the same core ¹⁰, enabling the seamless expansion without reinventing the wheel each time.

With these references and the detailed plan above, the GhostLink Diagnostic Tool is well-defined and ready to be built. It represents the “cold” initial version – lean, offline, and focused – yet sets the stage for a rich, modular toolset that can grow and adapt, always under the user’s sovereign control.

¹ ³ ¹³ Turn Your Raspberry Pi Into An OBD2

<https://projects-raspberry.com/turn-raspberry-pi-obd2/>

² How to Take Analog Readings on the Raspberry Pi : 5 Steps - Instructables

<https://www.instructables.com/How-to-Take-Analog-Readings-on-the-Raspberry-Pi/>

- 4 Using a Pi with browser and HDMI screen as a notification device/dashboard - #2 by Stiltjack - Configuration - Home Assistant Community
<https://community.home-assistant.io/t/using-a-pi-with-browser-and-hdmi-screen-as-a-notification-device-dashboard/594410/2>
- 5 14 25 GitHub - brendan-w/python-OBd: OBd-II serial module for reading engine data
<https://github.com/brendan-w/python-OBd>
- 6 7 8 9 10 Transcript.txt
<file:///file-H8museURvngFdLbewmd9rV>
- 11 12 15 16 17 18 19 20 21 22 23 24 scripts from research.txt
<file:///file-L9bqiT9TtjKoeZAVkjmdXe>